

KIẾN TRÚC PHIÊN VÀ CƠ CHẾ XOAY VÒNG TOKEN TRÊN REDIS

Hệ thống: Tomato Cinema
Microservices

Chuyên đề: Security & Redis Session
Management

Tiêu chuẩn: OAuth 2.0 Security BCP (RFC
6749)

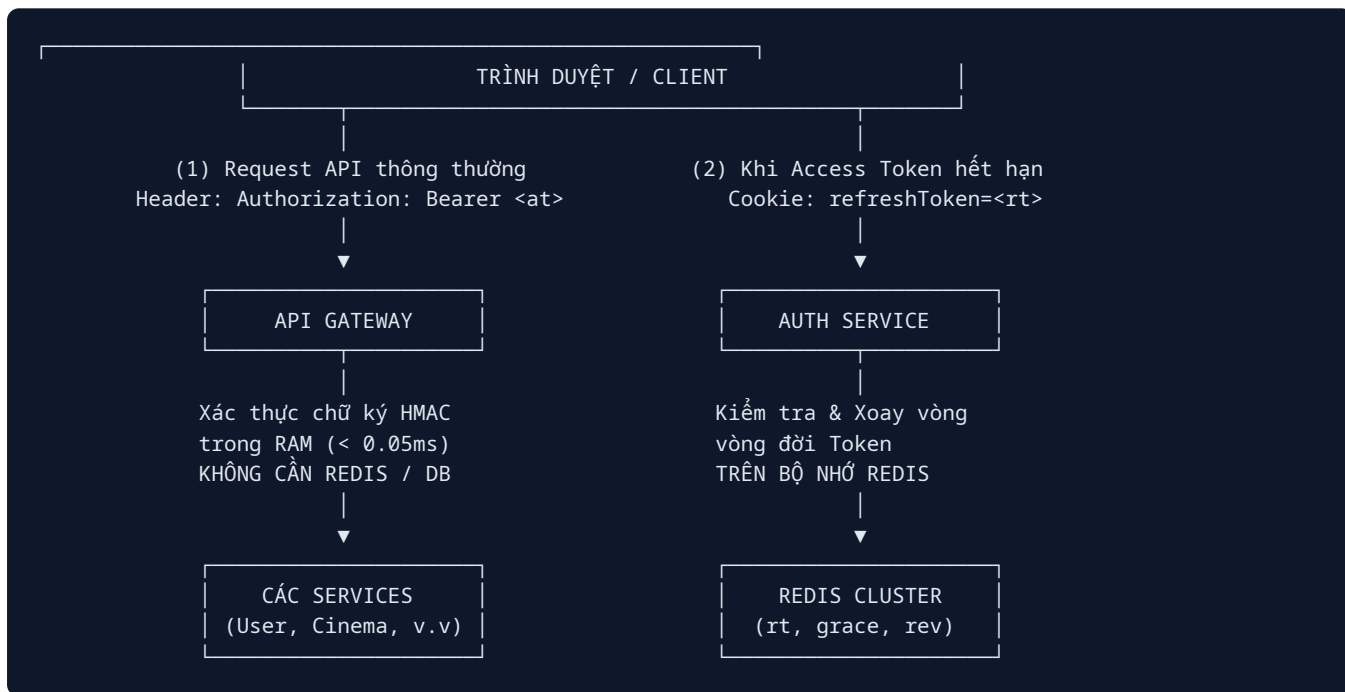
KIẾN TRÚC PHIÊN VÀ CƠ CHẾ XOAY VÒNG TOKEN TRÊN REDIS

Dự án: Tomato Cinema Microservices System Tác giả: Đội ngũ Kỹ thuật Tomato Cinema Tài liệu: Security & Architecture Deep-Dive Phiên bản: 1.0.0

1. TỔNG QUAN KIẾN TRÚC XÁC THỰC HYBRID

Hệ thống Tomato Cinema áp dụng mô hình xác thực lai (Hybrid Token Authentication):

- **Access Token là Stateless:** Không lưu trữ trong Redis hay Cơ sở dữ liệu.
- **Refresh Token là Stateful:** Được giám sát và kiểm soát vòng đời chặt chẽ trên bộ nhớ đệm Redis.



2. TẠI SAO ACCESS TOKEN KHÔNG NẰM TRONG REDIS?

Nhiều lập trình viên thường thắc mắc: "Tại sao tìm trong Redis không thấy Access Token?"

1. **Hiệu năng cực đại (Zero-I/O Verification):** * Nếu lưu Access Token vào Redis, mỗi request từ client (hàng chục nghìn req/giây) đều phải thực hiện truy vấn mạng tới Redis để kiểm tra hợp lệ. * Tại Tomato Cinema, Access Token được ký bằng thuật toán HMAC SHA-256 qua thư viện `@tomatocinema/passport`. API Gateway chỉ cần giải mã payload và xác thực chữ ký toán học ngay trong RAM của Node.js với độ trễ **< 0.05ms**. 2. **Thời gian sống ngắn (Short-lived):** * Access Token chỉ có thời hạn 1 giờ (3,600 giây). Nếu lỡ bị lộ, thời gian hacker có thể lợi dụng là rất ngắn. 3. **Giảm tải cho hạ tầng Cache:** * Giúp cụm Redis không bị phình to bởi hàng triệu token ngắn hạn, dành toàn bộ tài nguyên RAM cho các tác vụ quan trọng như Session, Seat Locking (khóa ghế) và Caching dữ liệu phim.

3. CƠ CHẾ 3 NHÓM KHÓA REFRESH TOKEN TRONG REDIS

Refresh Token có thời hạn dài (24 giờ đến 30 ngày) nên bắt buộc phải là **Stateful** để hỗ trợ tính năng đăng xuất tức thì, phát hiện xâm nhập và xoay vòng an toàn.

Mỗi Refresh Token khi lưu vào Redis đều được băm SHA-256 lấy 32 ký tự đầu làm `{fingerprint}` (dấu vân tay):

```
fingerprint = createHash('sha256').update(refreshToken).digest('hex').slice(0, 32);
```

> **Lợi ích:** Không lưu chuỗi token thô vào Redis. Ngay cả khi bộ nhớ Redis bị rò rỉ hoặc hacker dump được database, chúng cũng **không thể khôi phục ra chuỗi Refresh Token gốc** của người dùng.

Bảng cấu trúc 3 nhóm khóa:

Khóa Redis	Thời gian sống (TTL)	Mục đích kỹ thuật	Giá trị lưu trữ
<code>rt:{userId}:{fingerprint}</code>	24 giờ / 30 ngày	Active Session: Phiên Refresh Token hợp lệ duy nhất đang hoạt động.	<code>{"userId": "...", "createdAt": ...}</code> (~181 Bytes)
<code>rt_grace:{userId}:{fingerprint}</code>	30 giây	Grace Period (Cửa sổ Ân hạn): Lưu cặp token mới để xử lý Race Condition khi mở nhiều tab.	<code>{"accessToken": "...", "refreshToken": ...}</code> (~491 Bytes)
<code>rt_revoked:{userId}:{fingerprint}</code>	24 giờ	Revocation Tracking: Lưu vết token đã xoay vòng để phát hiện tấn công tái sử dụng (Reuse Detection).	Chuỗi <code>"1"</code> (~109 Bytes)

4. PHÂN TÍCH CASE STUDY THỰC TẾ TỪ REDIS COMMANDER

Dưới đây là dữ liệu thực tế thu thập từ giao diện Redis của tài khoản `bff2dc4d-d60e-444b-9e0a-54b7649b7838`:

```
1. NHÓM ACTIVE REFRESH TOKEN (rt):
- Key: rt:bff2dc4d-d60e-444b-9e0a-54b7649b7838:ec524309067f0a92ff98a91dc55a067f
- TTL: 23 h
- Dung lượng: 181 B

2. NHÓM CỬA SỔ ÂN HẠN (rt_grace):
- Key 1: rt_grace:...:0be92051a829f1381ea0f8b9580a4a9e | TTL: 25 s | 491 B
- Key 2: rt_grace:...:62592a2ad1cb8f07722522b326606990 | TTL: -2 s (vừa hết hạn)
- Key 3: rt_grace:...:a8793ff75be4a208fe64a70e14319797 | TTL: 11 s | 491 B

3. NHÓM LỊCH SỬ THU HỒI (rt_revoked):
- Key 1: rt_revoked:...:0be92051a829f1381ea0f8b9580a4a9e | TTL: 23 h | 109 B
- Key 2: rt_revoked:...:62592a2ad1cb8f07722522b326606990 | TTL: 23 h | 109 B
- Key 3: rt_revoked:...:a8793ff75be4a208fe64a70e14319797 | TTL: 23 h | 109 B
```

Hiện tượng gì vừa diễn ra trong hệ thống?

- Người dùng (hoặc trình duyệt) vừa thực hiện **3 lần xoay vòng token liên tiếp** trong vòng chưa đầy 30 giây qua:

* **Lần 1:** Dùng token `62592a2a...` $\rightarrow$ sinh ra token mới $\rightarrow$ `62592a2a...` vào grace (nay đã hết 30s nên TTL hiển thị `-2 s`) và ghi vết vào `rt_revoked`. * **Lần 2:** Dùng token `a8793ff7...` $\rightarrow$ sinh ra token mới $\rightarrow$ `a8793ff7...` vào grace (TTL còn 11s) và ghi vết vào `rt_revoked`. * **Lần 3:** Dùng token `0be92051...` $\rightarrow$ sinh ra token `ec524309...` (đang active trong `rt`, TTL 23h). Token `0be92051...` vào grace (TTL còn 25s) và ghi vết vào `rt_revoked`.

- Tại sao `rt_grace` có dung lượng 491 Bytes?

Vì `rt_grace` lưu trọn bộ cả cặp `{ accessToken, refreshToken }` mới dạng JSON, sẵn sàng phục vụ cho bất kỳ request nào gửi token cũ lên trong vòng 30s.

- Tại sao `rt_revoked` chỉ có 109 Bytes?

Vì `rt_revoked` chỉ lưu giá trị `"1"` đóng vai trò cờ đánh dấu (Tombstone Flag), giữ cho bộ nhớ Redis siêu nhẹ.

5. THUẬT TOÁN XOAY VÒNG VÀ PHÒNG VỆ CHUYÊN SÂU

```

public async rotate(refreshToken: string): Promise<{ accessToken: string; refreshToken: string }> {
    // 1. Thăm định HMAC SHA-256 của chuỗi Token
    const result = this.passportService.verify(refreshToken, 'refresh');
    if (!result.valid) throw new RpcException({ code: RpcStatus.UNAUTHENTICATED, details: result.reason });

    const { userId } = result;
    const fingerprint = this.computeFingerprint(refreshToken);

    // 2. BƯỚC CỨU TINH ĐA TAB (Grace Period 30s)
    const graceKey = `rt_grace:${userId}:${fingerprint}`;
    const graceSession = await this.redisService.get(graceKey);
    if (graceSession) {
        return JSON.parse(graceSession); // Trả ngay cặp token mới sinh trước đó
    }

    // 3. BƯỚC BẮT KÊ TRỘM (Token Reuse Detection)
    const revokedKey = `rt_revoked:${userId}:${fingerprint}`;
    const isReused = await this.redisService.get(revokedKey);
    if (isReused) {
        // Tín hiệu báo động đỏ: Có kẻ đang dùng lại token cũ ngoài 30s!
        await this.revokeAllRefreshTokens(userId);
        throw new RpcException({
            code: RpcStatus.UNAUTHENTICATED,
            details: 'Phát hiện tái sử dụng token bất thường. Toàn bộ phiên đã bị thu hồi!'
        });
    }

    // 4. Kiểm tra Active Session trong Redis
    const redisKey = `rt:${userId}:${fingerprint}`;
    const session = await this.redisService.get(redisKey);
    if (!session) throw new RpcException({ code: RpcStatus.UNAUTHENTICATED, details: 'Phiên không hợp lệ' });

    // 5. Cấp cặp Token mới
    const newTokens = await this.generate(userId);

    // 6. Cập nhật Redis nguyên tử bằng Pipeline
    const pipeline = this.redisService.pipeline();
    pipeline.del(redisKey); // Xóa active cũ
    pipeline.set(graceKey, JSON.stringify(newTokens), 'EX', 30); // Lưu grace 30s
    pipeline.set(revokedKey, '1', 'EX', 86400); // Lưu vết 24h
    await pipeline.exec();

    return newTokens;
}

```

6. HAI BÀI TOÁN KINH ĐIỂN ĐƯỢC GIẢI QUYẾT TRIỆT ĐỂ

6.1. Bài toán Race Condition khi Mở Nhiều Tab Trình Duyệt

- Tình huống:** Người dùng mở 5 tab xem phim trên Tomato Cinema. Sau 1 giờ, Access Token hết hạn. Trình duyệt gửi đồng thời 5 request `POST /auth/refresh`.
- Xử lý:**

* Tab đầu tiên đến Auth Service: Xoay vòng token thành công, lưu bản sao vào `rt_grace` trong 30s. * 4 Tab tiếp theo đến sau vài mili-giây: Cầm token cũ hỏi hệ thống `$\rightarrow$` Trúng ngay `rt_grace` `$\rightarrow$` Cả 4 tab đều nhận được token mới. * **Kết quả:** Không có bất kỳ tab nào bị gián đoạn hay bị văng ra màn hình đăng nhập.

6.2. Bài toán Chống Kẻ Gian Đánh Cắp Token (Replay Attack)

- Tình huống:** Kẻ tấn công đánh cắp được Refresh Token cũ trên đường truyền mạng.
- Xử lý:**

* Token cũ sau 30 giây đã hết hạn trong `rt_grace`. * Khi hacker cố dùng token này để refresh lấy Access Token mới `$\rightarrow$` Vướng ngay vào `rt_revoked`. * **Hệ thống phản ứng:** Ngay lập tức gọi `revokeAllRefreshTokens(userId)` quét sạch tất cả các session của `userId` đó trên Redis bằng lệnh `SCAN`. * **Kết quả:** Cả hacker lẫn người dùng đều bị đăng xuất. Tài khoản được bảo vệ 100% trước nguy cơ bị chiếm quyền âm thầm.

7. BẢNG SO SÁNH TỔNG KẾT

Tiêu chí	Access Token	Refresh Token
Bản chất	Stateless (Tự chứa thông tin)	Stateful (Quản lý trạng thái)
Nơi lưu trữ	Không lưu trên Server (Client giữ trong RAM)	Redis Cache (<code>rt</code> , <code>rt_grace</code> , <code>rt_revoked</code>)
Thời hạn sống	Ngắn (1 giờ)	Dài (24 giờ đến 30 ngày)
Vị trí Client	Header <code>Authorization: Bearer <at></code>	Cookie <code>HttpOnly: true</code> , <code>SameSite=Lax</code> , <code>Secure</code>
Chi phí thẩm định	Cực thấp (< 0.05ms, chỉ kiểm tra toán học trong RAM)	Trung bình (1 lần truy vấn Redis có pipeline)
Khả năng thu hồi	Hết hạn tự nhiên	Thu hồi tức thì bằng <code>DEL</code> trong Redis
Phòng vệ đặc thù	Thời gian sống ngắn hạn chế rủi ro	Token Rotation + Grace Period 30s + Reuse Detection